

Re: "In-app messaging: all five findings, answered"

Teamder — 23 August 2026. Verified against the live workspace after your deploy.

Thank you for the turnaround, and for §04's third gap — `package.json`'s `files` array omitting `react-native.config.js` is one we could not have seen, and it would have made the fix look done while shipping nothing.

Three things below: what we verified working, one correction to your §00, and **one finding that is still open** — the vocabulary fix does not reach the branch that records the click.

1. Verified working

- **Unknown actions are rejected loudly.** `'button:cta'` now returns `recorded: false`, `reason: "unknown_action"`, with the `expected` list. Exactly as described.
- **Our integration is correct against the new server.** We send `action: "click"` plus `clicked: true` / `clickedAt`, and the click records. Confirmed end to end on a production build: one button tap moved `clicks 4 → 5` and `dismissals 2 → 3` while `impressions` held at 10.
- `push_permission` — your correction is right and useful. The SDK does refresh on init, foreground and its own prompt; it was `registerPushToken()` specifically that did not, which is the path an app owning its push stack takes. We work around it client-side today by re-reading `getPushPermissionStatus()` after registration.

2. Correction to §00: our fix was necessary but was not sufficient

Your summary says the headline symptom is "fixed and live now — no client change required." That is true for the *server* half. It was not the whole cause.

After changing `'button:cta'` → `'clicked'`, clicks were still **zero**. The second cause was in the Android SDK we had vendored: its `TrackImpressionRequest` carried the interaction only in `action`, with no structured markers. We captured it off the wire from a release build:

```
--> POST /v1/in-app/track
{"action":"dismissed","anonymousId":"","campaignId":"","deliveryToken":""}
<-- 200
{"success":true}
```

No `in_app.dismissed` event was written. Your `main` already fixes this — the newer `InAppMessagingManager` maps the action onto the markers — so we re-vendored from `main` and the click landed immediately.

We mention it because **the published SDK still carries the old shape**. Anyone integrating from the released sources today gets an in-app channel that reports 0% CTR, and §00's "no client change required" would send them looking in the wrong place.

3. STILL OPEN: a recognised action with no markers records nothing

This is the same silent drop, in a narrower place, and the new `recorded: false` honesty does not cover it.

`resolveInAppAction()` is called, and its result is used **only** to compute `reportsDisplay`. The three branches that actually emit still key on the markers alone:

```
const actionKind = resolveInAppAction(impressionData.action); // line 1594
// ...used only in reportsDisplay...

if (impressionData.clicked) { emit('in_app.clicked', ...) }
if (impressionData.action === 'dismiss' || impressionData.dismissedAt) { ... }
if (impressionData.converted) { emit('in_app.converted', ...) }
```

So an action-only client that sends `'clicked'` — a value your own `expected` array advertises, and the exact value your SDK docs instruct (`Joryio.trackInAppImpression(message.id, 'displayed')` ... "and `'clicked'` / `'dismissed'` as the user acts on it") — is classified correctly as a click, excluded from `reportsDisplay` **because** it is a click, and then emitted by nothing.

Measured, against your live server, one campaign, four calls

SENT	RESPONSE	RECORDED
<code>action: "clicked"</code> , no markers	<code>{"success": true}</code>	no

SENT	RESPONSE	RECORDED
<code>action: "click" + clicked: true + clickedAt</code>	<code>{"success": true}</code>	yes
<code>action: "dismissed", no markers</code>	<code>{"success": true}</code>	no
<code>action: "button:cta"</code>	<code>{"success": true, "recorded": false, "reason": "unknown_action"}</code>	no (correct)

Rows 1 and 3 are the problem: a bare `{"success": true}`, no `recorded: false`, nothing written. Before your change an unknown action at least became a display — wrong, but *visible*. A recognised action with no markers is now silent.

A second, smaller bug in the same block

Line 1641 compares against the literal `'dismiss'`:

```
if (impressionData.action === 'dismiss' || impressionData.dismissedAt) {
```

Your own canonical spelling is `'dismissed'`, which does not equal `'dismiss'`. That branch is reachable by the singular alias only; every other accepted spelling — `dismissed`, `closed` — depends entirely on `dismissedAt` being set.

Suggested fix

`actionKind` already holds the answer; the branches just are not asking it:

```
if (impressionData.clicked || actionKind === 'click') { emit('in_app.clicked', ..
if (impressionData.dismissedAt || actionKind === 'dismiss') { emit('in_app.dismissed',
if (impressionData.converted || actionKind === 'convert') { emit('in_app.converted', ..
```

This is worth doing on the server rather than waiting for the SDK release, for the same reason you gave for the vocabulary itself: it repairs every already-shipped SDK version without anyone re-vendoring. With it, the published SDKs work as documented today, and §02's finding stops being a release blocker for new integrators.

If you would rather keep markers mandatory, then the honest alternative is to return `recorded: false, reason: 'missing_markers'` — the same shape as `unknown_action`. Either is fine. Silence is the thing to remove.

4. Your three asks

1. **Change one string** — done before your reply arrived; we send `'clicked'` and `'dismissed'`, typed as a union so a stray string no longer compiles, with a test that fails if any literal outside the vocabulary reaches the call.
2. **Tell you if we see** `recorded: false` — we will. So far the only one we have produced is the deliberate `'button:cta'` probe above.
3. **Re-vendor when the release lands and drop our local** `react-native.config.js` — will do. We are currently on `main` rather than the published package, for the reason in §2.

One request in return: when the SDK release lands, please say so somewhere we can poll. We vendor from source, so we have no npm version to watch, and §03 and §01 are both fixes we are carrying local workarounds for until then.